

Deep Learning

1. What is Hebbian learning? Explain its principle in biological neurons.

Hebbian learning is a classic rule in neural networks inspired by how human brains learn, summarized by psychologist Donald Hebb in 1949 as: "*Cells that fire together, wire together.*"

- **Principle in Biological Neurons:** When two connected neurons are activated at the same time repeatedly, the connection (synapse) between them strengthens. This means Neuron A becomes more efficient at triggering Neuron B in the future because their synchronized activity reinforces their pathway.

2. What is a threshold in an artificial neuron?

A **threshold** is a specific limit value that determines whether an artificial neuron should "fire" (activate) or stay inactive.

- **How it works:** The neuron calculates the total sum of its inputs multiplied by their weights. If this sum is greater than or equal to the threshold value, the neuron outputs a 1 (active); otherwise, it outputs a 0 (inactive).

3. What is a biological neuron? How does it relate to learning in Deep Learning?

A **biological neuron** is a nerve cell in the brain that receives information through branch-like structures called **dendrites**, processes the signals in the cell body (**soma**), and transmits electrical signals down a long fiber called the **axon** to other neurons.

- **Relation to Deep Learning:** Artificial neurons are mathematical copies of biological brain cells.
 - Dendrites → Input values (x)
 - Synaptic strength → Weights (w)
 - Cell firing limit → Threshold / Activation function

4. Describe the McCulloch–Pitts neuron model.

The **McCulloch–Pitts (MCP) neuron** is the first-ever mathematical model of a neuron, introduced by Warren McCulloch and Walter Pitts in 1943.

- **Mechanism:** It accepts binary inputs (0 or 1). Each input has a fixed weight—either **excitatory** (+1) or **inhibitory** (−1). The neuron adds up these weighted inputs. If the total sum is greater than or equal to a pre-set threshold value, the neuron fires an output of 1; otherwise, it outputs 0. It acts essentially like a logical switch (AND, OR).

5. Differentiate between the McCulloch–Pitts model and a biological neuron.

Feature	McCulloch–Pitts (MCP) Model	Biological Neuron
Weights / Learning	Has fixed weights; it cannot learn or adjust weights on its own.	Has adaptive weights (synaptic plasticity) that change and learn from experience.
Signal Type	Uses strictly binary signals (0 or 1).	Uses complex chemical and electrical signals (graded and action potentials).
Complexity	A very simple mathematical logic gate.	A highly complex, living biological cell with thousands of connections.

6. What is a Multilayer Perceptron (MLP)? Mention its basic layers.

A **Multilayer Perceptron (MLP)** is a type of feedforward artificial neural network composed of multiple layers of interconnected nodes (neurons). Unlike a basic single-layer network, an MLP can learn and solve complex, non-linear problems by passing data through intermediate processing layers.

Basic Layers of an MLP

1. **Input Layer:** Receives the original raw input data (features) and passes them forward without any mathematical alteration.
2. **Hidden Layer(s):** One or more layers located between the input and output layers. This is where the network extracts complex patterns, hidden features, and applies non-linear activation functions.
3. **Output Layer:** Receives the processed signals from the hidden layers and produces the final prediction, classification, or result.

7. Differentiate between a single-layer perceptron and a Multilayer Perceptron (MLP).

Feature	Single-Layer Perceptron (SLP)	Multilayer Perceptron (MLP)
Structure	Contains only an input layer and an output layer (has no hidden layers).	Contains an input layer, one or more hidden layers, and an output layer.
Problem Complexity	Can only solve simple, linearly separable problems.	Can solve complex, non-linear problems (like the XOR problem, image classification, etc.).
Learning Rule	Uses the basic Perceptron Learning Rule to update weights.	Uses Backpropagation with gradient descent to update weights across multiple layers.
Computation Power	Limited and very basic.	High processing power capable of handling deep learning tasks.
Example	Simple logical gates like AND or OR.	Recognizing handwritten digits, voice recognition, and sentiment analysis.

8. What is the purpose of an activation function in an MLP? Name any three activation functions.

Purpose: An activation function adds **non-linearity** to a neural network. Without it, stacking multiple layers together would just act like a single linear calculation, making it impossible for the network to learn complex, real-world patterns.

Three Common Activation Functions:

1. **ReLU (Rectified Linear Unit):** Outputs the input directly if it is positive; otherwise, it outputs zero.
2. **Sigmoid:** Squeezes input values into a range between 0 and 1 (useful for binary classification).
3. **Tanh (Hyperbolic Tangent):** Squeezes input values between -1 and 1 .

9. Why can an MLP solve the XOR problem while a single-layer perceptron cannot?

- **Single-Layer Perceptron (SLP):** The XOR logic problem is **non-linearly separable**, meaning it is impossible to draw a single straight line on a graph to separate the correct outputs. An SLP can only draw one straight line, so it fails.
- **Multilayer Perceptron (MLP):** Because an MLP has hidden layers, it can combine multiple lines or draw complex curved decision boundaries, successfully separating the non-linear data points required to solve XOR.

10. What is Gradient Descent? What is its objective?

Gradient Descent is an iterative optimization algorithm used to find the lowest point of a mathematical function.

- **Objective:** To find the optimal weights and biases for a neural network that minimize the **loss function** (error), thereby making the model's predictions as accurate as possible.

11. What is Backpropagation? Mention its main steps.

Backpropagation (backward propagation of errors) is the core algorithm used to train neural networks by calculating how much each weight contributed to the total error.

Main Steps:

1. **Forward Pass:** Pass input data through the network to generate a prediction.
2. **Calculate Loss:** Compare the prediction against the actual target using a loss function.
3. **Backward Pass:** Use the chain rule of calculus to compute the gradient (error derivative) of the loss with respect to each weight, moving backward from the output layer to the input layer.
4. **Weight Update:** Adjust the weights using Gradient Descent to reduce the error for the next iteration.

12. What is a loss function and what is its role in Backpropagation?

A **loss function** is a mathematical formula that measures the penalty or error by comparing a neural network's prediction against the true target value.

- **Role in Backpropagation:** It acts as the starting point for backpropagation. The error value calculated by the loss function provides the initial signal at the output layer, which is then sent backward through the network to guide weight adjustments.

13. What is Empirical Risk Minimization (ERM)?

Empirical Risk Minimization (ERM) is a learning principle in machine learning where a model is trained by minimizing the total error (risk) calculated specifically on a **training dataset** (the empirical data available).

- **Concept:** Instead of knowing the true distribution of all possible data in the real world, the algorithm optimizes its parameters to perform as well as possible on the observed training samples.

14. Differentiate between True Risk and Empirical Risk.

Feature	True Risk (Expected Risk)	Empirical Risk
Definition	The theoretical error a model makes over the entire, infinite population of all possible data (the real world).	The measured error of a model calculated specifically on a finite training dataset.
Accessibility	Unknown/Unreachable because we cannot see all possible real-world data.	Computable directly using the available training samples.
Goal	We want to minimize true risk so the model performs well on new data.	We minimize empirical risk during training because true risk is hidden.

15. What is regularization? How does it help prevent overfitting?

Regularization is a technique used to prevent overfitting by adding a penalty term to the loss function based on the complexity of the model (such as penalizing large weights).

- **How Overfitting Happens:** Overfitting occurs when a model memorizes the training data too closely—including random noise and minor details—which causes it to fail when tested on new, unseen data.
- **How it prevents overfitting:** It discourages the model from memorizing random noise or tiny details in the training data. By forcing the network to keep its weights smaller and smoother, it improves the model's ability to generalize well on unseen test data.

16. What is L1 Regularization (Lasso)? How does it affect the weights of a model?

L1 Regularization (also called Lasso) adds the sum of the absolute values of the model's weights as a penalty to the loss function.

- **Effect on weights:** It can drive less important weights down to **exact zero**. This effectively performs feature selection, removing useless features from the model entirely and creating sparse models.
- **Main Benefit:** Because it sets useless feature weights to zero, it acts as an automated feature selection tool. It completely removes irrelevant inputs from the model, resulting in a simpler, sparser, and more interpretable network.

17. What is the Vanishing Gradient problem in deep neural networks?

Definition: The vanishing gradient problem occurs during backpropagation in deep neural networks when the gradients (the error derivatives) become exponentially small as they travel backward from the output layer toward the input layers.

- **Why It Happens:** During backpropagation, the chain rule multiplies many small numbers (derivatives of activation functions like sigmoid or tanh) across many deep layers. Multiplying numbers less than 1 repeatedly causes the gradient value to shrink rapidly near zero.
- **Consequence:** Because the gradients approach zero in early layers, the weights in those layers barely update or stop learning entirely, preventing the deep network from training properly.

18. What is an Autoencoder? Explain its basic architecture.

An **autoencoder** is an unsupervised artificial neural network designed to learn how to compress data (dimensionality reduction) and then reconstruct it back as closely as possible to the original input.

Basic Architecture: It consists of three main parts sequentially linked:

Input Data → [Encoder] → [Latent Space / Bottleneck] → [Decoder] → Reconstructed Output

Description: An autoencoder consists of three sequentially connected parts:

1. **Encoder:** Takes the high-dimensional input data and compresses it into a lower-dimensional format.
2. **Latent Space (Bottleneck):** The middle layer that holds the compressed code, forcing the network to retain only the most important features while discarding noise.
3. **Decoder:** Takes the compressed representation from the bottleneck and expands it back to reconstruct the original input data.

19. What are the functions of the Encoder, Latent Space (Bottleneck), and Decoder in an Autoencoder?

- **Encoder:** Compresses the high-dimensional input data into a lower-dimensional representation.
- **Latent Space (Bottleneck):** The constrained middle layer that forces the network to retain only the most important features and patterns while discarding noise.
- **Decoder:** Takes the compressed code from the latent space and expands it back to reconstruct the original input format.

20. What is CNN feature extraction?

CNN feature extraction is the automated process in a Convolutional Neural Network where filter kernels slide across an image to detect and extract visual patterns—such as edges and corners in early layers, and textures, shapes, or complex objects in deeper layers—without requiring manual feature engineering.

21. What is convolution in a Convolutional Neural Network (CNN)? Explain its basic operation.

Definition: Convolution is the fundamental mathematical operation in a CNN where a small grid of weights called a filter slides across an input image to extract local visual patterns.

- **Basic Operation Steps:**

1. **Kernel Sliding:** A small filter (kernel) is placed over the top-left corner of the input image matrix.
2. **Element-wise Multiplication:** Each value in the kernel is multiplied by the corresponding overlapping pixel value of the image.
3. **Addition (Summation):** All of these individual multiplication products are added together along with a bias term to get a single total sum.
4. **Feature Map Generation:** This single value represents one pixel in the output **feature map**. The kernel then shifts by a set step size (stride) to repeat the process across the entire image.

22. What is a kernel (filter) in CNN? What is its role in feature extraction?

Definition: A kernel (also called a filter) is a small matrix of learnable weights (e.g., 3×3 or 5×5) used to scan across an input image.

- **Role in Feature Extraction:** As the kernel moves over the image, it acts as a pattern detector. Different kernels learn to look for specific visual elements:
 - **Early Layers:** Kernels detect low-level features like vertical edges, horizontal lines, and corners.
 - **Deep Layers:** Kernels combine those edges to detect mid-level textures and high-level complex shapes or object parts (like eyes, wheels, or faces).

23. What is a feature map? Explain how a feature map is generated using convolution.

Definition: A feature map is the filtered output matrix produced after applying a convolution kernel over an input image. It highlights the locations where specific features appear in the image.

- **How It Is Generated:**

1. The kernel slides across the input image row by row and column by column.
2. At every location, it performs **element-wise multiplication and addition** of the overlapping pixel values.
3. The resulting scalar numbers are collected and arranged into a new 2D grid, which forms the feature map. For example, a filter sensitive to vertical lines will produce a feature map with high values wherever vertical lines exist in the input image.

24. What is stride in CNN? How does changing the stride affect the output size?

Definition: Stride is the number of pixels by which the kernel shifts or moves across the input matrix during a convolution operation.

- **Effect on Output Size:**

- **Small Stride (e.g., Stride = 1):** The kernel moves one pixel at a time, resulting in a large output feature map that overlaps heavily and preserves fine details.
- **Larger Stride (e.g., Stride = 2 or more):** The kernel skips pixels as it moves, which drastically reduces the overlap. This results in a **smaller output feature map** (downsampling), reduces computational cost, and summarizes the image faster.

25. What is padding in CNN? Why is zero padding used?

Definition: Padding is the technique of adding extra rows and columns of pixels (typically filled with zeros, known as **zero padding**) around the outer borders of an input image or feature map before applying convolution.

- **Why Zero Padding Is Used:**

1. **Prevents Image Shrinking:** Without padding, every time a filter is applied, the output matrix shrinks (e.g., a 5×5 image with a 3×3 filter becomes 3×3). Padding allows the output to retain the same spatial dimensions as the input.
2. **Preserves Border Information:** Pixels in the center of an image are processed more often than pixels at the corners and edges. Padding gives border pixels extra context so the network doesn't lose valuable edge information.

26. Write the output-size formula of a convolution layer and explain the terms N, F, P and S.

Formula:

$$O = \frac{N - F + 2P}{S} + 1$$

Explanation of Terms:

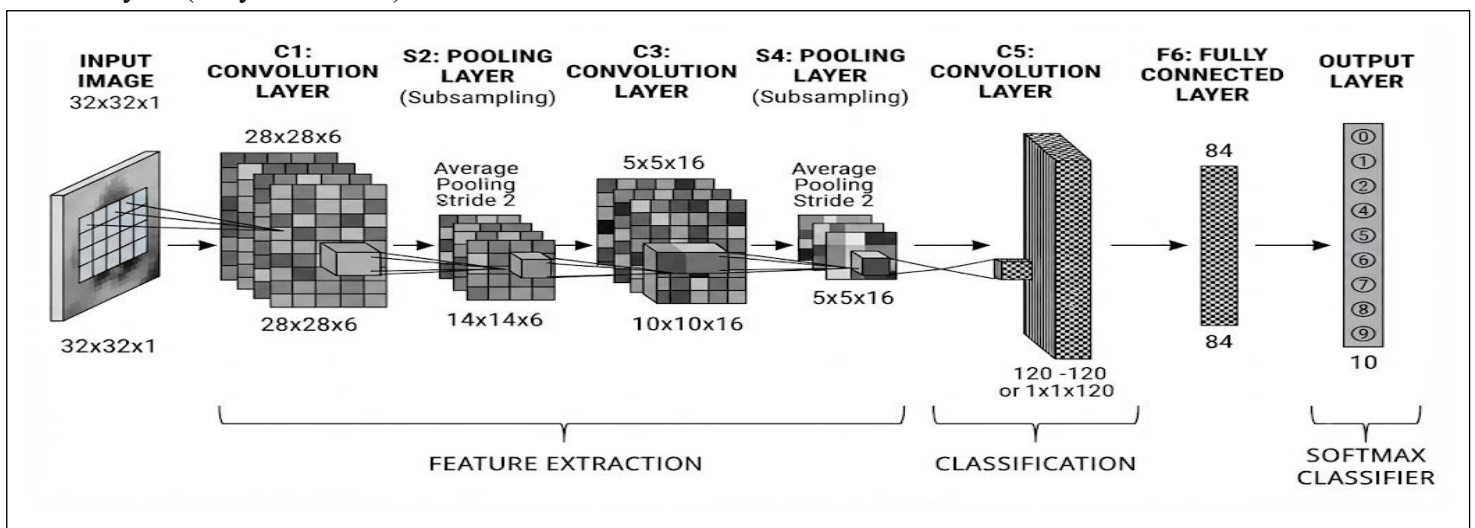
- **N (Input Size):** The spatial dimension (width or height) of the input image or feature map (assuming a square matrix).
- **F (Filter Size):** The spatial dimension (width or height) of the convolution kernel/filter.
- **P (Padding):** The number of rows or columns of pixels added to the border of the input (typically zero padding).
- **S (Stride):** The step size or number of pixels by which the filter shifts across the input matrix.

27. What are the main features and architecture of LeNet?

LeNet-5 (developed by Yann LeCun in 1998) is one of the earliest and most famous Convolutional Neural Networks, originally designed to recognize handwritten digits (such as ZIP codes and bank checks).

Main Features:

- **Local Receptive Fields:** Neurons look at small, local patches of the image rather than the whole image at once.
- **Weight Sharing:** Filters share the same weights across different parts of the image, drastically reducing the total number of parameters.
- **Alternating Layers:** Combines feature extraction layers (convolution and pooling) with classification layers (fully connected).



28. What is AlexNet? Explain its architecture and major innovations.

What is AlexNet: AlexNet is a landmark Convolutional Neural Network designed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton. In 2012, it won the ImageNet Large Scale Visual Recognition Challenge by a massive margin, proving the power of deep learning for computer vision and sparking the modern AI revolution.

Architecture: AlexNet consists of **8 layers** in total:

- **5 Convolutional Layers** (some followed by max-pooling layers) for feature extraction.
- **3 Fully Connected Layers** at the end for classification, ending with a Softmax output layer for 1,000 image categories.

Major Innovations:

1. **ReLU Activation:** Replaced slower traditional activation functions like sigmoid.
2. **Dropout Regularization:** Used to prevent overfitting in dense layers.
3. **GPU Training:** Split the model across two graphics cards to process massive data efficiently.
4. **Overlapping Pooling:** Used max-pooling layers where pooling windows overlapped, slightly reducing error rates.

29. Differentiate between LeNet and AlexNet.

Feature	LeNet-5 (1998)	AlexNet (2012)
Year	1998	2012
Dataset	MNIST (Handwritten digits, 10 classes)	ImageNet (1.2 million high-res images, 1,000 classes)
Input Size	Small (32×32 pixels)	Large (224×224 or 227×227 pixels)
Activation Function	Sigmoid or Tanh	ReLU (Rectified Linear Unit)
Pooling Type	Average Pooling	Max Pooling (with overlapping)
Parameters	Small (~60,000 parameters)	Massive (~60 million parameters)
GPU Usage	Trained on a single CPU	Trained across two GPUs in parallel

30. What are the important innovations of AlexNet? Explain the role of ReLU, Dropout and GPU training.

1. **Role of ReLU (Rectified Linear Unit):** AlexNet replaced traditional activations (sigmoid/tanh) with ReLU ($f(x) = \max(0, x)$). Its biggest advantage is that it solves the vanishing gradient problem, allowing gradients to flow easily during backpropagation. This caused network training to run **many times faster** than networks using sigmoid functions.
2. **Role of Dropout:** Dropout is a regularization technique where a random percentage of neurons are temporarily turned off ("dropped out") during each training step. This prevents neurons from depending too heavily on each other (co-adaptation), forcing the network to learn robust features and **preventing overfitting** on large models.
3. **Role of GPU Training:** Because ImageNet was massive and AlexNet had 60 million parameters, a standard CPU would have taken weeks or months to train it. The creators split the network architecture across **two NVIDIA GPUs**, allowing parallel processing that cut down training time to just a few days and proved that graphics cards are essential for deep learning.

31. Numerical question to practice: Given an input size, filter size, padding and stride, calculate the output feature-map size

Convolution output size formula:

$$O = \frac{N - F + 2P}{S} + 1$$

Example 1: Without Padding ($P = 0$)

- Given Data: Input size (N) = 5; Filter size (F) = 3; Padding (P) = 0; Stride (S) = 1
- Calculation:

$$O = \frac{5 - 3 + 2(0)}{1} + 1$$
$$O = \frac{2}{1} + 1 = 2 + 1 = 3$$

- Final Output Size: 3×3

Example 2: With Zero Padding ($P = 1$)

- Given Data: Input size (N) = 5; Filter size (F) = 3; Padding (P) = 1; Stride (S) = 1
- Calculation:

$$O = \frac{5 - 3 + 2(1)}{1} + 1$$
$$O = \frac{5 - 3 + 2}{1} + 1$$
$$O = \frac{4}{1} + 1 = 4 + 1 = 5$$

- Final Output Size: 5×5

THANK YOU